# API7:2023 Server Side Request Forgery

| Threat agents/Attack vectors | Security Weakness | Impacts |
| --- | --- | --- |
| API Specific : Exploitability **Easy** | Prevalence **Common** : Detectability **Easy** | Technical **Moderate** : Business Specific |
| Exploitation requires the attacker to find an API endpoint that accesses a URI that's provided by the client. In general, basic SSRF (when the response is returned to the attacker), is easier to exploit than Blind SSRF in which the attacker has no feedback on whether or not the attack was successful. | Modern concepts in application development encourage developers to access URIs provided by the client. Lack of or improper validation of such URIs are common issues. Regular API requests and response analysis will be required to detect the issue. When the response is not returned (Blind SSRF) detecting the vulnerability requires more effort and creativity. | Successful exploitation might lead to internal services enumeration (e.g. port scanning), information disclosure, bypassing firewalls, or other security mechanisms. In some cases, it can lead to DoS or the server being used as a proxy to hide malicious activities. |

## Is the API Vulnerable?

Server-Side Request Forgery (SSRF) flaws occur when an API is fetching a remote resource without validating the user-supplied URL. It enables an attacker to coerce the application to send a crafted request to an unexpected destination, even when protected by a firewall or a VPN.

Modern concepts in application development make SSRF more common and more dangerous.

More common - the following concepts encourage developers to access an external resource based on user input: Webhooks, file fetching from URLs, custom SSO, and URL previews.

More dangerous - Modern technologies like cloud providers, Kubernetes, and Docker expose management and control channels over HTTP on predictable, well-known paths. Those channels are an easy target for an SSRF attack.

It is also more challenging to limit outbound traffic from your application, because of the connected nature of modern applications.

The SSRF risk can not always be completely eliminated. While choosing a protection mechanism, it is important to consider the business risks and needs.

## Example Attack Scenarios

### Scenario #1

A social network allows users to upload profile pictures. The user can choose either to upload the image file from their machine, or provide the URL of the image. Choosing the second, will trigger the following API call:

```
POST /api/profile/upload_picture

{
  "picture_url": "http://example.com/profile_pic.jpg"
}
```

An attacker can send a malicious URL and initiate port scanning within the internal network using the API Endpoint.

```
{
  "picture_url": "localhost:8080"
}
```

Based on the response time, the attacker can figure out whether the port is open or not.

### Scenario #2

A security product generates events when it detects anomalies in the network. Some teams prefer to review the events in a broader, more generic monitoring system, such as a SIEM (Security Information and Event Management). For this purpose, the product provides integration with other systems using webhooks.

As part of a creation of a new webhook, a GraphQL mutation is sent with the URL of the SIEM API.

```
POST /graphql

[
  {
    "variables": {},
    "query": "mutation {
      createNotificationChannel(input: {
        channelName: \"ch_piney\",
```

```
        notificationChannelConfig: {
          customWebhookChannelConfigs: [
            {
              url: \"http://www.siem-system.com/create_new_event\",
              send_test_req: true
            }
          ]
          }
      }){
        channelId
    }
    }"
  }
]
```

During the creation process, the API back-end sends a test request to the provided webhook URL, and presents to the user the response.

An attacker can leverage this flow, and make the API request a sensitive resource, such as an internal cloud metadata service that exposes credentials:

```
POST /graphql

[
  {
    "variables": {},
    "query": "mutation {
      createNotificationChannel(input: {
        channelName: \"ch_piney\",
        notificationChannelConfig: {
          customWebhookChannelConfigs: [
            {
              url: \"http://169.254.169.254/latest/meta-data/iam/security-
credentials/ec2-default-ssm\",
              send_test_req: true
            }
          ]
        }
      }) {
        channelId
      }
    }
  }
]
```

Since the application shows the response from the test request, the attacker can view the credentials of the cloud environment.

## How To Prevent

- Isolate the resource fetching mechanism in your network: usually these features are aimed

to retrieve remote resources and not internal ones.

- Whenever possible, use allow lists of:
  - Remote origins users are expected to download resources from (e.g. Google Drive, Gravatar, etc.)
  - URL schemes and ports
  - Accepted media types for a given functionality
- Disable HTTP redirections.
- Use a well-tested and maintained URL parser to avoid issues caused by URL parsing inconsistencies.
- Validate and sanitize all client-supplied input data.
- Do not send raw responses to clients.

## References

### OWASP

- Server Side Request Forgery
- Server-Side Request Forgery Prevention Cheat Sheet

### External

- CWE-918: Server-Side Request Forgery (SSRF)
- URL confusion vulnerabilities in the wild: Exploring parser inconsistencies, Snyk